

@ MICROSOFT HQ 2025

Become immediately effective with PowerShell

Jason Helmick – PowerShell PM

Sean Wheeler – PowerShell Documentarian

Steven Judd – PowerShell MVP

Level: Beginner

Session Survey

- Your feedback is very important to us
- Please take a moment to complete the session survey found in the mobile app
- Use the QR code or search for “Converge360 Events” in your app store
- Find this session on the Agenda tab
- Click “Session Evaluation”
- Thank you!



Introduction

- Who are we?
- What is PowerShell?
 - It's a command shell & programming language!
- Versions and installation
- Command discovery and help



Lab 1 – Install PowerShell & Help

1. Install PowerShell 7 (if you can)
2. Get help for a command before help is installed:

```
Get-Help Get-Command -Full
```

3. Run Update-Help to install the latest help on your computer.

```
Update-Help -Force
```

This requires that the shell is running with elevated privileges (use the **Run as Administrator** option). NOTE: You will get error messages about failing to download help for specific modules. This is a known issue.

4. Get help after help is installed, run:

```
Get-Help Get-Command -Full
```

Notice the difference in the amount of help available.

Help, docs, & the community

- Get-Help in depth
 - Command discovery using help
- Understanding command syntax
- Docs on Microsoft Learn
- Get help from the community



Lab 2 - Command discovery and problem solving

Command discovery

1. Can you find any cmdlets capable of converting other cmdlets' output into HTML?
2. How many cmdlets are available for working with processes? (Hint: remember that cmdlets all use a singular noun.)
3. Is there a way to shutdown a remote computer?

Problem solving

1. How would you find the Background Intelligent Transfer Service?
2. What is the current status of the Background Intelligent Transfer Service?
3. Is the Background Intelligent Transfer Service set to start automatically?
4. How would you stop the Background Intelligent Transfer Service?

Modules & pipeline

Modules

- Finding and installing modules
- Importing modules
- Discovering new commands

Connecting commands in a pipeline

- What's the pipe and what does it do?
- Sorting, selecting, filtering, and formatting
 - Filter left, format right
- Difference between Out-*, Write-*, and Format-* cmdlets
 - Out-* - Output to something other than the console
 - Write-* - Writes a PowerShell output stream
 - Format-* - Formats the output for display (format objects)



Lab 3 – Modules & Pipeline

Modules

1. What modules are available on your system?
2. What is the latest version of the **Pester** module available in the PowerShell Gallery?
2. What properties are available on the objects returned by `Get-Process`? How can you display all of them?

Using the pipeline

1. What's difference between the following commands?

```
Get-ChildItem -File | Format-Table Name,  
Length | Sort-Object Length
```

```
Get-ChildItem -File | Format-Table Name,  
Length | Get-Member
```

```
Get-ChildItem -File | Sort-Object Length |  
Format-Table Name, Length
```

3. Run the following command and compare the output:

```
Get-Process pwsh
```

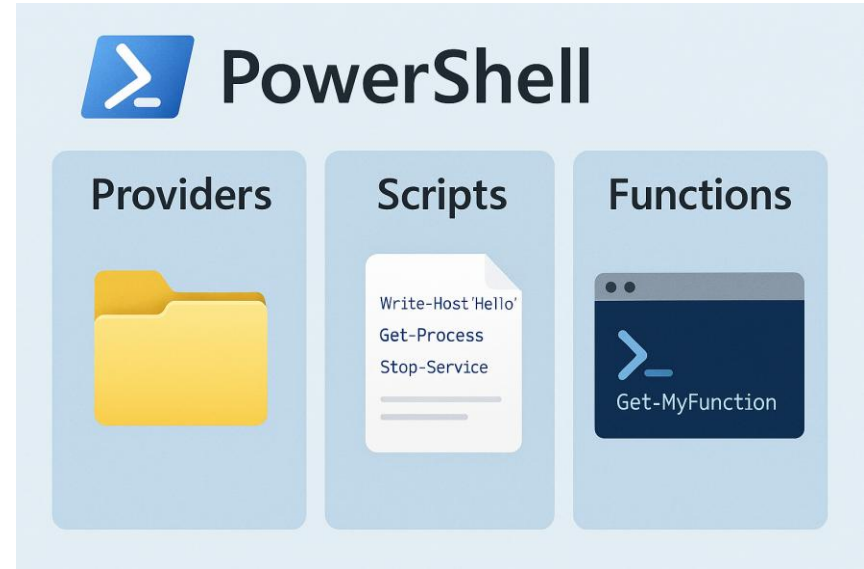
```
Get-Process pwsh | Format-Table
```

```
Get-Process pwsh | Format-List
```

```
Get-Process pwsh | Select-Object -Property *
```

Providers, functions & scripts

- The magic of PowerShell providers
- Turn one-liners into scripts
 - Batch files vs. scripts
 - Using parameters
- Turn scripts into functions
 - Advanced functions

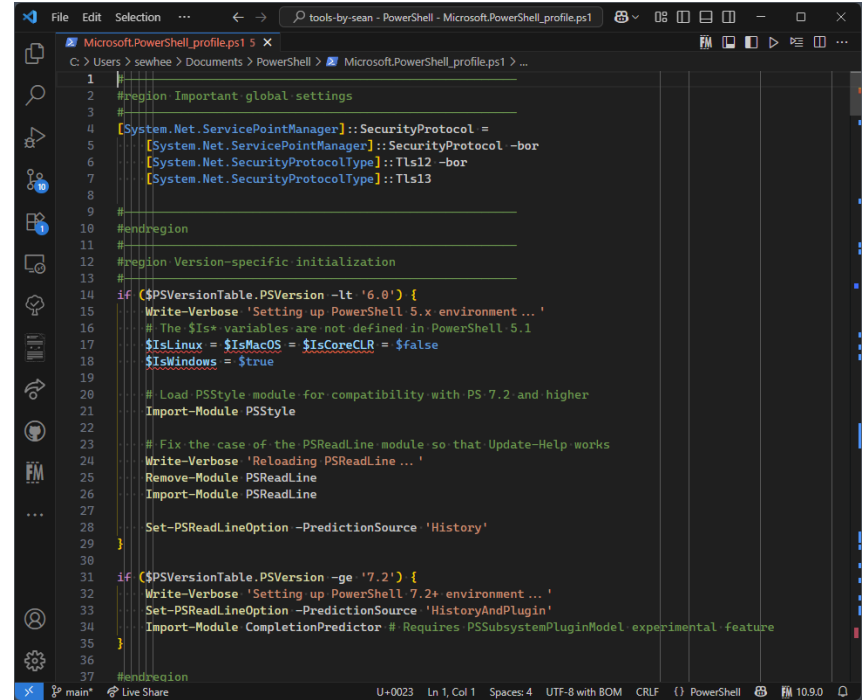


Lab 4 - Functions & Scripts

Create a function called `Get-AssetInfo` that retrieves information about the computer system and BIOS. The function should accept a parameter for the computer name, defaulting to the local computer if none is provided. It should return a custom object with properties such as **ComputerName**, **Manufacturer**, **Model**, **BIOSVersion**, and **SerialNumber**.

Profiles

- Intelligent shell features
 - Tab completions & command predictors
 - Color settings
 - Dynamic help & key bindings
 - Feature configuration
- Profile scripts

A screenshot of a code editor window showing a PowerShell profile script. The window title is "Microsoft.PowerShell_profile.ps1". The script content includes comments for "Important global settings" and "Version-specific initialization". It sets security protocols for ServicePointManager, checks the PowerShell version (6.0 or 7.2), and configures the PSReadLine module with various options like PredictionSource and CompletionPredictor. The script is written in a dark-themed editor with syntax highlighting.

```
1  
2 #region Important global settings  
3  
4 [System.Net.ServicePointManager]::SecurityProtocol =  
5 [System.Net.ServicePointManager]::SecurityProtocol -bor  
6 [System.Net.SecurityProtocolType]::Tls12 -bor  
7 [System.Net.SecurityProtocolType]::Tls13  
8  
9  
10 #endregion  
11  
12 #region Version-specific initialization  
13  
14 if ($PSVersionTable.PSVersion -lt '6.0') {  
15     Write-Verbose 'Setting up PowerShell 5.x environment...'  
16     # The $Is* variables are not defined in PowerShell 5.1  
17     $IsLinux = $IsMacOS = $IsCoreCLR = $false  
18     $IsWindows = $true  
19  
20     # Load PSStyle module for compatibility with PS 7.2 and higher  
21     Import-Module PSStyle  
22  
23     # Fix the case of the PSReadLine module so that Update-Help works  
24     Write-Verbose 'Reloading PSReadLine...'  
25     Remove-Module PSReadLine  
26     Import-Module PSReadLine  
27  
28     Set-PSReadLineOption -PredictionSource 'History'  
29 }  
30  
31 if ($PSVersionTable.PSVersion -ge '7.2') {  
32     Write-Verbose 'Setting up PowerShell 7.2+ environment...'  
33     Set-PSReadLineOption -PredictionSource 'HistoryAndPlugin'  
34     Import-Module CompletionPredictor # Requires PSSubsystemPluginModel experimental feature  
35 }  
36  
37 #endregion
```

Lab 5 – Create a profile

1. Create a profile script
2. Add the function you created in Lab 4
3. Configure PSReadLine key bindings for the following keys:

Key chord	Function
Enter	ValidateAndAcceptLine
Alt+a	SelectCommandArgument
F1	ShowCommandHelp
Alt+h	ShowParameterHelp

PowerShell Remoting

Remoting enable commands

- WMI & Window RPC-enabled commands in PS 5.1

Remoting over WinRM

- One-to-one or Interactive
- One-to-many

Remoting over SSH

- One-to-one or Interactive

Windows Compatibility feature

- Deserialized objects



Q&A

Stump the
Chumps

Ask the
Experts



Session Survey

- Your feedback is very important to us
- Please take a moment to complete the session survey found in the mobile app
- Use the QR code or search for “Converge360 Events” in your app store
- Find this session on the Agenda tab
- Click “Session Evaluation”
- Thank you!

